

ease process, those first rings will be controlled by the existing release-tools tooling.

mermaid

flowchart LR

subgraph releasetools [Release-tools coordinator pipeline]

direction LR

gstgcny[GSTG\n Canary] --> gstgcnyqa[QA]

gstgcnyqa --> gprdcny[GPRD\n Canary]

gprdcny --> gprdcnyqa[QA]

gprdcnyqa --> bakingtime[1hr\n baking time]

bakingtime --> promotion[..\n manual\npromotion]

promotion --> gstg[GSTG\nMain] --> gstgqa[QA]

promotion --> |delay 30m|gprd[GPRD\nMain]

end

subgraph pdm [Post Deployment Migrations]

direction LR

pdmgstg[GSTG] --> pdmgstgqa[QA] --> pdmgprd[GPRD]

end

subgraph Ring Deployment

direction LR

ring0[Ring 0] --> ring0qa[QA]

ring1[Ring 1]

ring2[Ring 2]

%%% invisible link to help content layout

ring1 ~~~ ring2

end

pkg((Auto-deploy\npackage\ntagging)) --> releasetools

%% ring deployment hooks

gstgcnyqa --> ring0

ring0qa --> bakingtime

pdmgprd ---> |pkg graduation|ring2

promotion --> |delay 30m| ring1

Future - Only Cells

For the sake of completeness we are also describing a potential future scenario where the legacy infrastructure is decommissioned and all of our users are migrated to a new cell.

This is subject to change as the project evolves, as an example, the future of our staging environment will likely be affected by the development of Cells, but here we leave it in place to reduce the scope of this document.

mermaid

flowchart LR

subgraph releasetools [Release-tools coordinator pipeline]

direction LR

gstgcny[GSTG\n Canary] --> gstgcnyqa[QA]

bakingtime[1hr\n baking time] --> promotion[..\n manual\npromotion]

```

    promotion --> gstg[GSTG\nMain] --> gstgqa[QA]
end
subgraph Post Deployment Migrations
    direction LR
    pdmgstg[GSTG] --> pdmgstgqa[QA] --> pdmgprd[Ring 1]
end
subgraph Ring Deployment
    direction LR
    ring0[Ring 0] --> ring0qa[QA]
    ring1[Ring 1]
    ring2[Ring 2] --> ring3[Ring 3] --> ringN[Ring N]
end
pkg((Auto-deploy\npackage\ntagging)) --> releasetools
%% ring deployment hooks
gstgcnyqa --> ring0
ring0qa --> bakingtime
pdmgprd ---> |pkg graduation|ring2
promotion ---> |delay 30m| ring1

```

The changes from the co-existing scenario are the following ones:

1. New rings exist after Ring 2. The progression of packages is coordinated by the ring-deployment engine enforcing the gates on each ring.
1. The release-tools coordinator pipeline only manage rings, there is no longer a concept of canary and main stage in production.
1. The post deployment migration pipeline controls the execution of the migrations in Ring 1 (Subject to further discussion that are not necessary at this stage of the project).

Reference materials

- Cell 1.0 blueprint
- The merge request for this blueprint
- Delivery Point of View on Cells
- GitLab.com deployment process before Cells

Goals and Non-Goals

Goals

- Limit the increase in cognitive load for the release manager; in doing so, we defined the perimeter as the clear handover point where a package is no longer a release manager's responsibility.
- Limit the blast radius of failures by partitioning the cell cluster into rings, automated validation occurs between each ring.
- Ensure deployments are reliably automated
- Ensure automatic handling of failed deployments
- Provide observability into package rollouts and deployments

Non-Goals

- Extending release-tools to take ownership of the Cell Application Deployments. A smaller, more specific piece of software will allow us to keep the tooling focused on one job.
- Introduce major changes related to Release Management
- Lifecycle management of Cells
- Management of routing traffic to/from Cells
- Individual component deploys

Requirements

Before we can integrate Cells to our deployment pipeline, we need a few items immediately:

1. The router should exist, it must be HA, and have an independent deployment pipeline
 - This is required for appropriate testing. As noted below, we'll need a QA cell to direct a deployment to for which QA will execute tests against. A router will need to route QA tests to the appropriate Cell.

1. Assets Deployment

- This already exists today for .com. Today this is handled via HAProxy, but with Cells, the routing layer will become the responsible party to redirect assets in a similar fashion.
- If assets are chosen to be managed differently, this changes both how Delivery need to deploy said assets in order to provide as close to Zero-Downtime Upgrades as possible, and configuration to the Cell installation to support routing to assets properly.

1. Feature Flags

- We are assuming that the current Feature Flags workflows and tooling will just work on the Legacy Cell and that Cells will not be affected.
- The use of feature flags to mitigate incidents is limited to only the Legacy Cell.
- Tooling may need to mature to ensure that Cells do not drift for long periods of time with feature flags. This ensures that customers have a similar experience if their work expands across Cells and that we as operators of .com need not worry about version drift and the implications of code differing behind the feature flag.
- Further guidance, documentation will need to be developed for this area. Engineers shouldn't care what cell an organization is a part of. Thus Feature Flag toggles abstract away the need for engineers to care.

Proposed plan of action

From a delivery perspective not much changes between the 3 proposed Cells iterations (1.0, 1.5, and 2.0). The split is an iterative approach based on cutting the scope of the features available for Organizations bound to a given Cell. From a deployment point of view, it should be possible to have multiple Cells from the first iteration so we have to figure out a roadmap to get there that is independent from the Cell architecture version.

Iterations

Cells 1.0

The intent in this iteration is to focus our efforts on building and integrating our own tooling that builds and manages Cells. The following milestones, and their exit criterion, are a collaborative effort of the Platforms section and spans across many teams.

1. The Dedicated technology stack expansion:

- Instrumentor and AMP support GCP
 - A cell is defined as a reference architecture in Instrumentor
 - 1. Control Plane for Cells - Cell Cluster Coordinator
 - Switchboard is currently leveraged by Dedicated but is not an appropriate tool for Cells. We should evaluate the capabilities of other tooling created by Dedicated, amp and instrumentor, to determine how they could be integrated into a deployment workflow.
 - Implement Cell deployment converging the entire infrastructure of the cell (current dedicated capability)
 - Implement the concept of Rings: initially only Rings 0 and 2
 - 1. First Cell: the QA Cell in Ring 0
 - Build integration with our current tooling to perform deployments to the QA cell via the Coordinator
 - The QA Cell runs it's own QA smoke tests
 - The QA Cell is updated in parallel with the production canary stage: QA cell failures are considered soft and do not block autodeploy
 - 1. Control Plane for Cells - Individual dashboards and alerting
 - observability is at least on par with the legacy infrastructure
 - alerting is at least on par with the legacy infrastructure
 - 1. First Customer Cell: Ring 2
 - release-tools can graduate a package after the PDM execution
 - the Coordinator can manage Ring 2 deployments
 - 1. Support for multiple Cells
 - the Coordinator can converge multiple cells in the same Ring to the desired version
- > - Limitations:
- > - all Cells will be in the same ring, Ring 2
 - > - Rollbacks are possible but require downtime to achieve on all Cells

Cells 1.5 and 2.0

The following features can be distributed between Cell 1.5 and 2.0, they are all improving the operational aspects and we should prioritize them as we learn more about operating Cells.

- 1. Control Plane for Cells - Additional rings
 - Cells can be spread over multiple rings
 - Deployment to the next ring starts automatically after the current ring converged
 - Emergency brake: ability to block package rollout to the next ring
- 1. The QA Cell becomes a blocker for auto-deploy
- 1. Control Plane for Cells - Cluster dashboards and alerting
 - A dashboard should indicate what package is expected for any given Cell and Ring deployment
 - Any cell not running the desired version should be easily visible and alert if not converged in a reasonable amount of time
 - Deployment health metrics to block package rollout inside a ring (z-score on the four golden signals?)
- 1. The Post Deploy Migration (PDM) step of deployments needs to be segregated from the application deployment to ensure we have the ability to perform rollbacks on Cells.
 - Without this capability, a Cell must suffer downtime in order for a rollback to complete successfully. This is disruptive and should not be considered a wise solution.
 - The separation of the PDM on the Legacy Cell already functions as desired. Thus our

Legacy Cell will have rollbacks as an option to mitigate incidents.

1. Modified tooling that enables us to target only Deploying the GitLab application. Currently the destined tooling to be leveraged employs a strategy where the entire installation is converged. This includes the infrastructure and the version of GitLab which creates a lengthy CI pipeline and long running jobs.

1. Automated Rollbacks - if a deployment fails for any reason, a rollback procedure should be initiated automatically to minimize disruption to the affected Cell. We should be able to use a health metric for this.

The focus here is productionalizing what has been built and cleaning up areas of tech debt incurred during the MVP stage of the first iteration.

Mindmap

mermaid

```
%%{init: {'theme':'default'}}%%
```

mindmap

root((Cells Deployment))

Core concepts ·

Current .com infra is the Legacy Cell

It is possible to have multiple Cells in Cell 1.0

Cell isn't an HA solution

Cells talks to the Topology Service

Prerequisites ··

router

HA solution

independent deployment

Dedicated

Support GCP

Cell reference architecture

Decisions ·

Ring style deployment

Ring 0: Current Canary + a new QA Cell

Ring 1: Current Main stage

Ring 2+: New Cells for customers

The Perimeter

Marks the handover point between Release Managers and Team:Ops

Inside: Ring 0 and 1

Outside: Ring 2+

Running PDM inside the perimeter graduates a package

A graduated package is a valid candidate for the monthly release

We are not porting staging to Cell

A new QA Cell will validate the package without affecting users

Procedures needing upgrades ·

Rollbacks

auto-deploy rollout

Post Deployment Migration

Deployment health metrics

Risk area ..

There is no dogfooding in Cell 1.0 and 1.5

Deployment coordinator and Cell Cluster Coordinator

In the context of auto deploy we have an external coordinator pipeline, inside the release-tools project, that takes care of orchestrating package generation and rollout invoking the specific tool for each job.

In today's GitLab.com infrastructure, deployments are executed by specific tools (deployer and gitlab-com/k8s-workloads) that can be independently operated by SRE and Release Managers, with the introduction of the Cell cluster we will face new operational challenges like a simple cluster overview, package rollout status, feature flag configuration, provisioning and deprovisioning.

The GitLab Dedicated stack features its own method of controlling installs of GitLab, primarily through a slew of tools, Switchboard, Amp, and Tenctl. The use of Switchboard is not geared towards Cells and thus cannot be leveraged. Other tooling such as Instrumentor and Amp may have a place or modifications to enable them to be more portable for usage between both the Dedicated team and Cells. We'll need to evaluate these tools, their interactions with Cells, and how we may leverage them. Pending how the work is scheduled, this may be a highly collaborative effort with team members working closely across team boundaries to ensure requirements are met during the initial period or MVP for Cells.

In this paragraph we describe an ideal interaction where a data store is updated with a desired version to be deployed, and a Cell Cluster Coordinator is created to support Cell deployments.

In Cell 1.0, inside the perimeter, we will have a single Cell, the QA Cell.

We should expand release-tools to command some-tool to perform a Cell update on demand.

```
plantuml
@startuml
participant "release-tools" as rt
participant "Cell Cluster Coordinator" as sb
participant "AMP Cluster" as AMP
collections "Cells" as cells
participant QA
```

```
rt -> sb: update QA Cell
note right: In parallel with canary stage rollout
sb -> AMP: schedule deployment job
AMP -> cells: QA Cell: version rollout
cells --> AMP
AMP --> sb
sb --> rt
rt --> QA: test QA Cell
note over rt,QA: It does not replace canary QA
```

```
QA -> cells: run tests
cells --> QA
QA --> rt
@enduml
```

As we mentioned before, when we run post-deployment migrations in Ring 1, release-tools will mark that version as graduated and thus be capable to rollout outside of the perimeter.

Cell Cluster Coordinator will be leveraged to help coordinate automated version upgrades to further rings with automated checks before and after deployments to ensure we are deploying to the correct cells of a desired ring and validate instances are healthy before and after deployments, rolling back in the face of failure, and alerting the appropriate teams as necessary.

```
plantuml
@startuml
participant "Cell Cluster Coordinator" as sb
participant "AMP Cluster" as AMP
collections "Cells" as cells
```

```
loop
sb -> sb: pull ring-version mapping
opt version mismatch
sb -> AMP: schedule configure job
AMP -> cells: CellIX: version rollout
cells --> AMP
AMP --> sb
opt the ring now run the same version
sb -> sb: promote version to the next ring
end
end
end
@enduml
```

Procedures

Auto-Deploy

Auto-deploy shall continue to work as it does today as our Legacy Cell is equivalent to our legacy .com infrastructure. Thus our existing procedures related to auto-deploy can still be continued to be leveraged. Think hot-patching, rollbacks, auto-deploy picking, the PDM, the existing auto-deploy schedule, etc. A new procedure will be added to ensure that release-tools knows to trigger a deployment after a PDM is executed to the next Ring. Currently release-tools doesn't understand anything related to Ring Deployments, this is functionality that will need to be added.

- Auto-deploy is limited to Rings 0 and 1:
 - Ring 0 contains a QA Cell plus the canary stage of the .com infra

- Ring 1 contains main stage of the .com infra - this is the cut off for release tools
- All cells will deploy the same way; this eliminates needing to deal with differing deployment technologies
- release-tools will interact with the Coordinator to pilot the deployments to Ring 0 as part of its coordinator pipeline
- Release-tools must be able to graduate a package:
 - A graduate version of GitLab is any auto-deploy version which has a successful deploy onto the Main Stage of Production and the Post Deploy Migration (PDM) has completed.
 - This could mean we expect to see a single package deploy each day to our Cells. Currently, the PDM is only run 1 time per day. Note that there are exceptions to this rule.
 - This will enable us to use our existing procedures to remediate high severity incidents where application code may be at fault.
 - We do not want to run official released versions of GitLab as these are produced far slower than auto-deploys thus we risk missing SLA's on incident response. In the cell architecture, most issues should be found in the Legacy Cell and fixed prior to being deployed to any Cell.
 - We'll need new procedures, runbooks, and documentation such that when a problem is found through manual testing, we have some ability to halt deployments of what may be labeled a graduated package from actually being deployed.
 - It would be wise to track these failure cases as realistically, QA should be finding issues to enable us to run a automated deployments.

Note that currently, some smaller components deploy themselves to the .com infrastructure. Notably, Zoekt, Container Registry, and Mailroom, have their own cadence of providing newer versions to .com. This aspect will not be carried over into Cells, as currently, the tooling we'll leverage does not allow a segregation of components to enable this functionality. Instead, we'll rely on the current defined versions as specified in the default branch which built the auto-deploy package. This mimics how our releases are accomplished and thus should carry over well with Cells.

Rollbacks

Long term, we should aim to modify the deployment tooling such that Cells are provided a grace period to enable each of them to be able to be safely rolled back in the event of a deployment failure, or mitigating a failure that is noticed post deployment. Currently for the legacy .com or the Legacy Cell, we hold the PDM to execute 1 time per day at the discretion of Release Managers. The tooling that performs deployments to Cells currently do not have a way to NOT run the PDM, thus no there does not currently exist a way to rollback without inducing downtime on a particular Cell. Procedures and tooling updates will be required in this area.

Hot patching

Hot patching is one source of our ability to mitigate problems. If we rely on graduate versions, the hot patcher has no place for Cells. It could still be leveraged for our Legacy Cell, however. Though, it would be wise if we can eliminate hot patching in favor of safer deployment methodologies.

> For reference, we've only hot patched production 1 time for year 2023.

Deployment Health Metrics

Currently we do not automate a deployment to the Main stage of the .com legacy infrastructure, or the Legacy Cell. In order to reduce operational overhead we should be able to rely on existing metrics which form a health indicator for a given installation and automatically trigger a deployment at the appropriate time. This deployment health indicator would also need to be carried into each of our cells. Tooling that triggers a deployment at various rings should be made aware to continue or halt a deploy given the status of earlier rings and the health state of the next target ring.

Feature Flags

Feature Flags are discussed in [data-stores83](#).

Package Rollout Policy

We have an implicit procedure driven by our current use of auto-deploys. This will become more prominent with Cells. As implied in various formats above, auto-deploy shall operate relatively similarly to how it operates today. Cells becomes an addition to the existing release-tools pipeline with triggers in differing areas. When and what we trigger will need to be keenly defined. It is expected that Cells only receive graduated versions of GitLab. Thus, we'll leverage the use of our Post Deployment Migration pipeline as the gatekeeper for when a package is considered graduated. In an ideal world, when the PDM is executed successfully on the Legacy Cell, that package is then considered graduated and can be deployed to any outer ring. This same concept is already leveraged when we build releases for self managed customers. This break point is already natural to Release Managers and thus is a good carry over for Cell deployments.

We should aim to deploy to Cells as quickly as possible. For all Cells that exist in a single ring, we should have the ability to deploy in parallel. Doing so minimizes the version drift between Cells and reduces potential issues. If the version drifts too greatly, auto-deploy shall pause itself and an investigation into the reason why we are too far behind begins. Ideally we know about this situation ahead of time. We should aim to be no greater than 1 graduate package behind our PDM. Thus the expectation is that for every PDM, is a deployment to our Cells, every day. There are days which the PDM is skipped. We'll need to evaluate on a case-by-case basis why the PDM is halted to determine the detriment this will incur on our Cell deployments.

Rings outside of the perimeter are self-managed by the orchestration engine. Once release-tools graduates a package it can forget about it. The orchestration engine will converge the desired GitLab version to all Cell in Ring 2, the first ring outside of the perimeter, and move to next ring only when all Cells converged.

Select Helm Chart Updates

Currently, Instrumentor uses the default behavior of the GitLab Environment Toolkit to infer which version of our Helm Chart shall be utilized.

This is not suitable for Cells as .com currently leverages unpublished versions of our Helm chart for testing or dogfooding prior to release.

To keep feature parity in Cells, we'll need to add this capability in Dedicated Stack.

A blueprint to discuss and drive toward implementation of this feature has been proposed in Dedicated's team blueprints Enabling Prerelease Helm Chart Selection

FAQ

Will Developers see indicators on MR's as they are deployed to various Cells?

No. Our current labeling schema is primarily to showcase that the commit landed in production, the PDM successfully executed, which signals to us that the observed commit is safe for being placed in a release for self-managed customers. Being that after we reach this point, issues with a package should be minimal, there's no need to update issues/MR's with the status as we move forward into our many Rings of deployments. Developers should not need to care what version is deployed to what Cell.

A P1/S1 issue exists, how do we mitigate this on Cells?

Cells are still a part of .com, thus our existing bug and vulnerability

SLA's for remediation apply. We can deploy whatever we want to Cells so long as it's considered graduated. If a high priority issue comes about, we should be able to freely leverage our existing procedures to update our code base and any given auto-deploy branch for mitigation, and maybe after some extra rounds of testing, or perhaps a slower roll out, we can deploy that auto-deploy package into our cells. This provides us with the same mitigation methods that we leverage today. The problem that this causes is that there could exist some code that may not have been fully vetted. We can still rely on rollbacks in this case and revisit any necessary patch for the next round of auto-deployments and evaluate the fix for another attempt to remediate our cells.

What changes are expected from a Developers perspective

Release and Auto-Deploy procedures should largely remain the same. We're shifting where code lands. Any changes in this realm would increase the most the closer we are to Iteration 2.0 when various environments or stages to GitLab begin to change.

All tiers but one have a failed deploy, what triggers a rollback of that package for all cells?

This depends on various characteristics that we'll probably want to iterate on and develop processes for. Example, if we fail on the very first cell on the first Tier, we should investigate that cell, but also ensure that this is not systemic to all cells. This can only be handled on a case-by-case basis. If we reach the last tier and last cell and some failure would occur, there should be no reason to rollback any other cell as enough time should have passed by for us to catch application failures.

What happens with self-managed releases?

Theoretically not much changes. Currently we use Production, or .com's Main

Stage as our proving grounds for changes that are destined to be releasable for self-managed. This does not change as in the Cellular architecture, this notion for this exists in the same place. The vocabulary changes, in this case, a graduated package is now considered safe for a release.

What happens to PreProd

This instance specifically tests the hybrid installation of a GitLab package and Helm chart when we create release candidates. It's our last step prior to a release being tagged. This is not impacted by the Cells work. Though we may change how preprod is managed.

What happens with Staging

Staging is crucial for long term instance testing of a deployment alongside QA. Hypothetically staging could completely go away in favor of a deployment to Tier 0. Reference the above Iteration 3 {+TODO add proper link+}

What happens to Ops

No need to change. But if Cell management becomes easy, it would be prudent to make this installation operate as similar as possible to avoid overloading operations teams with unique knowledge for our many instances.

This same answer could be provided for the Dev instance.

SECTION: engineering/architecture/design-documents/cells/infrastructure/diff-between-dedicated.md

{{< engineering/design-document-header >}}

Existing Reads

1. Cell Iteration, specifically Cell 1.0
1. GitLab Dedicated
1. GitLab Dedicated technical documentation

GitLab Dedicated Diff

GitLab Dedicated had a chance to start from a blank slate and only run stable services. GitLab.com is running beta features or auxiliary that GitLab Dedicated doesn't have too.

Advanced Search

What is GitLab Dedicated Doing:

Dedicated relies on the GET tooling to provision AWS OpenSearch for tenants. The application is automatically configured by GET to configure the application to leverage

this feature.

What is GitLab.com doing right now:

GitLab.com leverages two items, one is under heavy development by group::Global Search. We have an Elasticsearch cluster which provides application level search capabilities for the vast majority of our customer base.

This is currently used for code search as well as searching anything else capable of being searched inside of the GitLab application.

The Global Search team are testing the use of Zoekt as an addition to eventually replace code search capabilities.

What will GitLab.com Cells do:

Accompanying issue in Dedicated

Capacity Planning

What is GitLab Dedicated Doing:

Tamland is used.

What is GitLab.com doing right now:

Tamland is used.

What will GitLab.com Cells do:

Tamland will be leveraged the same.

The work for Dedicated is complete.

Minor work may need to be accomplished to enable for Cells

Redis

What is GitLab Dedicated Doing:

Dedicated leverages GCP Memorystore product for Redis.

This work is a work in progress, but is supported by the GitLab Environment Toolkit.

What is GitLab.com doing right now:

At the .com scale, we have approximately 13 deployments of Redis all with targeted configurations pending their responsibility and role.

These Redis deployments also differ between standard Redis deployments with replicas and Redis Clusters.

We have a mix and match based on prioritized work that resolved various pain points as we scaled this service to suite our needs.

What will GitLab.com Cells do:

As Cells are a far smaller installation of GitLab, there should be no need to expand our Redis infrastructure as vast as we have for .com.

The chosen reference architecture should deploy a sufficiently sized Redis deployment to suit our needs.

Observability into the behavior of Redis when our first set of Cells receive customer traffic will need to be monitored closely to determine what areas of improvement we need to bring in.

Secret Management

What is GitLab Dedicated Doing:

Dedicated leverages Google Secret Manager for Secret Management.

Two types of secrets are leveraged in Dedicated, secrets which are unique per tenant and secrets which are shared per environment.

The latter being used strictly for management of tenants, such as with Amp.

Thus each GitLab installation has their own set of secrets and some configuration items are stored using that cloud providers secrets service.

Instrumentor, the Dedicated tenant provisioning tool is aware of and shares only the necessary secrets between various stages.

What is GitLab.com doing right now:

.com leverages KMS already for simplistic items for as well as a wrapper for secrets management for Chef runs on Virtual Machines.

Most secrets are pushed into HashiCorp Vault and various chunks of our infrastructure use this service for items that are shared between Virtual Machines and our Kubernetes installations.

What will GitLab.com Cells do:

The provisioning secrets and shared infrastructure secrets will be managed in Hashicorp Vault, both to be used strictly for management of tenants (Amp, CI, ...).

Each cell will have their own set of unique secrets which will be managed in Google Secret Manager.

The Kubernetes secrets will be synced from Google Secret Manager by the External Secrets operator.

Being discussed.

HAProxy

What is GitLab Dedicated Doing:

HAProxy is not used.

Instead we rely on the ingresses that are deployed as part of the GitLab Helm Chart which leverages the cloud load balancer to direct traffic to the appropriate underlying resources.

Each tenant has their own installation, thus a segregated endpoint for which customer traffic is routed too.

The deployments to handle frontend workloads are handled by a single service, known as webservice which is managed and deployed with our Helm Chart.

Further below, we mention the introduction of CloudFlare in the future, where this will eventually provide various WAF capabilities without the need for HAProxy.

What is GitLab.com doing right now:

HAProxy sits behind CloudFlare and behind a set of Google Load Balancers. The primary purpose is to provide traffic routing to target clusters for frontend traffic, and provides a large swath of rules for traffic management. Some of this management are throttling or request blocking based on various criterion. We have a few sets of HAProxy fleets that handle dedicated endpoints, such as `registry.gitlab.com`, the Pages endpoint, and CI traffic.

We leverage an advanced feature of our Helm Chart to deploy a multitude of frontend webservices, these are split into groups that serve target traffic. HAProxy is configured to divide these groupings of traffic with a myriad of rules. These groups are known today as `api`, `git`, `internal-api`, `web` and `websockets`.

What will GitLab.com Cells do:

GitLab Cells introduces a new layer where requests need to be routed. It is slated that the routing-service will intercept all requests to direct clients to the appropriate cluster. This layer is above that of our current HAProxy setup. Behind the routing-service we plan to leverage existing Ingress methods deployed as part of the GitLab Helm Chart. Requests will then go directly to the Ingress configurations of our frontend. We'll need to evaluate the various traffic rules which are configured in HAProxy to determine if they can be set inside of CloudFlare as firewall rules where necessary and able. Ideally, HAProxy goes away in Cells Architecture.

The Helm Chart is used to deploy the frontend Pods, only a single one will be deployed, mimicking what Dedicated does today.

Assets Routing

What is GitLab Dedicated Doing:

Assets are served by the frontend service, known as the webservice. webservice Pods deployed by the GitLab Helm Chart.

What is GitLab.com doing right now:

Assets are deployed into a Google Cloud Storage bucket as part of the deployment procedure owned by Team Delivery. Assets are then served by a specific routing configuration that is managed by CloudFlare. This removes our webservice deployment from needing to service static assets and allow those services to focus their load on processing requests.

What will GitLab.com Cells do:

The deployment of assets shall not change. This process will remain the same with CloudFlare.

Cloudflare

What is GitLab Dedicated Doing:

CloudFlare is not involved yet.

Work is in progress to bring a WAF solution to Dedicated.

What is GitLab.com doing right now:

CloudFlare is our first line of defense to provide us native and custom Firewall, DNS, and traffic routing management.

What will GitLab.com Cells do:

Our use of CloudFlare will technically expand with the use of the routing-service.

The .com entrypoint does not change, there should be no change to this service outside of the addition of the routing-service.

Container Registry

What is GitLab Dedicated Doing:

Dedicated leverages our Helm Chart to deploy and configure the GitLab Registry Service.

The Registry Service is backed by a storage bucket managed by the GitLab Environment Toolkit.

What is GitLab.com doing right now:

.com Leverages the same GitLab Helm chart to deploy and configure the Container Registry service.

The backing storage is configured by our own terraform mechanism in the config-mgmt repository.

Container Registry uses PostgresSQL Database for online garbage collection where the database migrations are managed by a Kubernetes Job with possibility to manually rollback.

The Container Registry also uses Redis as a caching layer.

What will GitLab.com Cells do:

Being Discussed

Mail Delivery

What is GitLab Dedicated Doing:

work in progress. See Issue 2481

- Outgoing: It is slated that we leverage the AWS SES product and geared towards outgoing email.

- Incoming: Incoming email is unsupported.

What is GitLab.com doing right now:

- Outgoing: Mail is handled by a third party, Mailgun.
- Incoming: Email is handled by the GitLab application with webhook delivery method which watches for email sitting on a configured inbox and directing this appropriately to Sidekiq workers.

What will GitLab.com Cells do:

Outgoing: Continue use Mailgun because we have a strong relationship with the provider, and no desire to change.

The Dedicated Provisioner can support Mailgun/Third-party outgoing mail gateways with minimal configuration change.

Incoming: Being discussed

PostgreSQL

What is GitLab Dedicated Doing:

A single RDS instance is deployed.

What is GitLab.com doing right now:

To handle the scale of some functions of our GitLab Rails codebase, the database was split into three PostgreSQL Clusters.

The main cluster which handles most of the data storage, the ci which handles all the CI related data, and finally, the embedding database.

We are also evaluating to further decompose to another PostgreSQL cluster.

In front of each database we have a set of PgBouncer as a connection pooler.

What will GitLab.com Cells do:

What we'll do is being worked on in a separate blueprint

Embedding Database

What is GitLab Dedicated Doing:

GitLab Dedicated is not using the embedding database, it's still in the experimental phase.

What is GitLab.com doing right now:

A PostgreSQL cluster deployed running pgvector.

The embedding data is to vectorize GitLab content (issues, epics, merge requests) and store that data in a place where it can be used for search, recommendations, anomaly detection, classification, backlog cleanup, deduplication, and other AI adjacent things.

What will GitLab.com Cells do:

Embedding Database is still considered experimental we will not support this in Cells to reduce scope for Cells 1.0.

GitLab Pages

What is GitLab Dedicated Doing:

Just recently released!

What is GitLab.com doing right now:

GitLab Pages is an active feature of .com.

What will GitLab.com Cells do:

We will not enable GitLab Pages for Cells 1.0 so it's out of scope at the moment.

VM Configuration Management

What is GitLab Dedicated Doing:

Management of the VMs is all handled by GitLab Environment Toolkit.
It uses Terraform for provisioning the machines and Ansible to configure the VM.

What is GitLab.com doing right now:

.com leverages Chef for configuration management with integrations into Vault for secrets management and a complex role structure built into Chef for managing configurations.

What will GitLab.com Cells do:

Reuse the GitLab Dedicated tooling, meaning that chef will be removed.

Disaster Recovery

What is GitLab Dedicated Doing:

Reference existing documentation:

- RTO/RPO from backups
- Recovery Guides

What is GitLab.com doing right now:

Reference existing documentation:

- Runbooks

What will GitLab.com Cells do:

A blueprint will be created at a later point in time.

Feature Flags

What is GitLab Dedicated Doing:

Feature Flags are not heavily leveraged and actively discouraged.

An override mechanism exists for special use-cases.

Note that this is a business decision and not a technical limitation.

What is GitLab.com doing right now:

We have a fairly complex process to allow engineers to rollout changes safely.

What will GitLab.com Cells do:

ChatOps will eventually be expanded upon to support Cells.

Epic 12797 has been created to provide any added functionality.

No work will occur for Iteration Cells 1.0.

Deployment

What is GitLab Dedicated Doing:

All components which feed into a tenant install are versioned.

Per contract, a maintenance window is defined and agreed upon with a customer that provides an opportunity for any maintenance, whether it be a version upgrade, change to infrastructure, or configuration change to the application is performed.

The tooling leveraged by this runs through extensive testing beforehand on non-production tenants.

What is GitLab.com doing right now:

A dedicated team, Delivery is responsible for a set of tooling which manages and deploys new versions of GitLab a multiple times per day.

Multiple stages, Canary and Main, are leveraged to help with risk management of deployments.

What will GitLab.com Cells do:

Delivery remains responsible for initial development of this.

The current blueprint that surrounds this work introduces a concept of Ring deployments to manage risk.

Subnets

What is GitLab Dedicated Doing:

Dedicated uses a static list of IP CIDRs that intentionally overlap for all tenants.

Subnets are configurable in the case of a need from a customer.

What is GitLab.com doing right now:

A very tightly and carefully managed dance is performed to ensure documentation, terraform code, and VPC peering all work in a cohesive manner across various GCP projects, and environments.

What will GitLab.com Cells do:

Being discussed

Kubernetes

What is GitLab Dedicated Doing:

Managed by the GitLab Environment Toolkit, a single Kubernetes Cluster is spun up.
Managed by the GitLab Helm Chart, an entire GitLab installation is deployed.
Instrumentor contains the tooling for the observability stack that is installed into the same cluster.

What is GitLab.com doing right now:

Clusters are manually configured with Terraform through the config-mgmt repository.
Deployments of anything to these clusters are managed through at least three repositories that deploy various tools required to manage, maintain, and observe the GitLab application workloads that are deployed.

What will GitLab.com Cells do:

Being discussed

SRE root access to machines

What is GitLab Dedicated Doing:

VMs:

Managed by Ansible: A mixture of SSH (internal link) with the use of Identify-Aware Proxy.
This must be used with the Break glass procedure (internal link).

Kubernetes:

kubectl: Break glass procedure (internal link).

What is GitLab.com doing right now:

VMs:

- VMs managed by Chef: Users and SSH keys are managed by Chef and require a bastion to be able

to SSH inside of machines with root access.

- Rails Console: We use Teleport to spin up a rails console for read-only access.
- psql: We use Teleport

Kubernetes:

- kubectl: A bastion server where the keys are managed by Chef and then setting up gcloud
- GKE VMs: We use Google's OS Login to access the nodes.

What will GitLab.com Cells do:

VMs:

- Managed by Ansible: Same as GitLab Dedicated
- Rails Console: Being discussed
- psql: Being discussed

Kubernetes:

- kubectl: Same as GitLab Dedicated
- GKE VMs: Same as GitLab Dedicated

Observability

What is GitLab Dedicated Doing:

An observability stack which consumes the same set of metrics we leverage for .com are bundled into Instrumentor and deployed to the observability stack installed into a tenant's Kubernetes Cluster.

This includes all the appropriate rules for alerting, paging, provides dashboards, and forecasting.

Some metrics are lacking, but are being worked on to bring full coverage where lacking.

There does not exist a global view of metrics for all tenants.

Logging is managed by way of using AWS OpenSearch for AWS tenants, and Google Cloud Logging for GCP tenants.

What is GitLab.com doing right now:

A large installation of the Prometheus and Mimir across a multitude of GCP projects are managed through various means.

We leverage the Runbooks repository to configure all of our Dashboards, Alerts, and Pages.

We inherently are provided a global view as our Grafana installation talks to a large Mimir configuration which contains data via remote-write from all necessary environments.

Logs are managed by way of fluentd on both our Virtual Machines and Kubernetes clusters sending data to PubSub which are then brought into Elasticsearch where Kibana is used for viewing.

Some services dump too much data, such as CloudFlare, GKE, and HAProxy, where we rely on

Google's Logging solution, either Stackdriver, or BigQuery.

What will GitLab.com Cells do:

Being discussed

Camoproxy

What is GitLab Dedicated Doing:

Unused.

What is GitLab.com doing right now:

go-camo is deployed by a custom Helm chart. go-camo used to serve images from HTTP based resources to HTTPS.

What will GitLab.com Cells do:

Being discussed

Certificates

What is GitLab Dedicated Doing:

GitLab Dedicated uses cert-manager, Let's Encrypt and NGINX to issue and manage certificates.

What is GitLab.com doing right now:

GitLab.com uses Cloudflare for certificates and mixture of cert-manager and GCP certificates for auxiliary services such as Grafana.

What will GitLab.com Cells do:

For the GitLab.com we will still use Cloudflare so we can continue using the certificates from Cloudflare.

osquery

What is GitLab Dedicated Doing:

It doesn't install osquery.

What is GitLab.com doing right now:

We have a chef cookbook to manage our osquery installation.

What will GitLab.com Cells do:

Tooling will need to be developed that is compatible with deployment methods being leveraged to

deploy to the Cell architecture.

What will be covered by osquery:

osquery would cover all the Virtual Machines; some of the examples today covered are bastion nodes, HA Proxy nodes, etc.

Why we require osquery:

1. Visibility: Notably, we still have very little visibility over what is happening inside our VMs and Kubernetes Clusters.
1. Compliance Requirements: Compliance requires us to have clear insights into actions in environments like Kubernetes or the Legacy VMs.
1. Incidents Investigation: Incidents reported in the past are left with few missing investigations due to a lack of detection and investigation of the commands executed before and after the malicious commands.

Wiz Runtime Sensor

What is GitLab Dedicated Doing:

Wiz is not used.

What is GitLab.com doing right now:

Wiz Runtime Sensor is a new tool being introduced into .com. Wiz Runtime Sensor is a lightweight eBPF-based agent which listens to syscalls and generates the findings on the suspicious actions executed by the applications/workloads. Follow [Wiz Runtime Internal Handbook Page](#) for more details.

As of this writing, it has not been deployed into the production environment and is still being tested in our staging environment. Currently, it is deployed using the Helm chart

What will GitLab.com Cells do:

If this tool is to be leveraged, we will need to recreate a method of deployment suitable for Cells.

The current deployment method being leveraged will not suffice for Cellular architecture.

What will be covered by Wiz Runtime Sensor:

Wiz Runtime Sensor would be deployed in the Kubernetes clusters as a DaemonSet. DaemonSet would add the sensor to all the nodes; Wiz Runtime Sensor would observe the syscalls for any malicious actions like malicious script executed, sensitive files accessed/modified, and container escape attempts.

Why we require Wiz Runtime Sensor:

1. Visibility: Notably, we still have very little visibility over what is happening inside our VMs and Kubernetes Clusters.

1. Compliance Requirements: Compliance requires us to have clear insights into actions in environments like Kubernetes or the Legacy VMs.

1. Incidents Investigation: Incidents reported in the past are left with few missing investigations due to a lack of detection and investigation of the commands executed before and after the malicious commands.

SECTION: engineering/architecture/design-documents/cells/infrastructure/disaster_recovery.md

{{< engineering/design-document-header >}}

Terms used

1. Legacy Cell: GitLab.com SaaS (the current GitLab.com deployment). It is called legacy because different tooling is used to operate it.

1. Cells: A set of infrastructure components that run independently and isolated from each other.

1. Global Service: A service to keep global uniqueness, manage database sequences across the cluster, and help classify which resources belong to which Cell.

1. Routing Service: The Routing Service depends on the Global Service and is for managing routing rules to different cells.

1. RTO: [Recovery Time Objective]

1. RPO: [Recovery Point Objective]

1. WAL: [Write-ahead logging]

Goals

Cells 1.0 is the first iteration of cells where Cells be operated independently of the Legacy Cell.

The Cells 1.0 target is to use Geo for Disaster Recovery which is the current DR strategy for GitLab Dedicated.

After Cells 1.0, we intend to leverage [Next Gen Scalable Backup and Restore] for restoring data from backups.

This document focuses only on defining the strategy for recovering Cells.

It does not cover recovering the Global Service, Routing Service, Legacy Cell, or any other external service.

Disaster Recovery for Cells creates a fork in our existing recovery process for the GitLab.com SaaS because cells are provisioned with different tooling.

It's possible we will have different RTO/RPO targets for Cells compared to the Legacy Cell, however the goal is to meet or exceed our Legacy Cell RTO/RPO targets listed below.

RTO/RPO Targets

NOTE:

FY25 targets have not yet been validated on the Legacy Cell.

Zonal Outages:

	RTO	RPO
Primary Cell (current)	2 hours	1 hour
Primary Cell (FY25 Target)	<1 minute	<1 minute
Cells 1.0 (without the primary cell)	unknown	unknown

Regional Outages:

	RTO	RPO
Primary Cell (current)	96 hours	2 hours
Primary Cell (FY25 Target)	48 hours	<1 minute [^object-storage]
Cells 1.0 (without the primary cell)	unknown	unknown

Disaster Recovery Overview

NOTE:

The services below are taken from the [Cells 1.0 Architecture Overview].

Zonal

Zonal recovery refers to a disaster, outage, or deletion that is limited in scope to a single availability zone.

The outage might affect the entire zone, or a subset of infrastructure in a zone.

Service	Zonal Disaster Recovery	Estimated RTO	Estimated RPO
GitLab Rails	All services running in a cell are redundant across zones. There is no data stored for this service.	<=1 minute	not applicable
Gitaly Cluster	Gitaly Cluster consists of a single SPOF (single point of failure) node and remains so for Cells 1.0. It requires a restore from backup in the case of a zonal failure.	<=30 min	<=1 hr for snapshot restore until WAL is available for restore.
[^blueprint-dr]			
Redis Cluster	Redis is deployed in multiple availability zones and be capable of recovering automatically from a service interruption in a single zone.	<=1 minute	<=1 minute
PostgreSQL Cluster	PostgreSQL cluster is deployed in multiple availability zones and be capable of recovering automatically from a service interruption in a single zone. A small amount of data-loss might occur on failover.	<=1 minute	<=1 minute

Regional

Regional recovery refers to a disaster, outage, or deletion that is limited in scope to an entire region.

The outage might affect the entire region, or a subset of infrastructure that affects more than one zone.

Service	Regional Disaster Recovery	Estimated RTO	Estimated RPO
GitLab Rails	All services running in a cell are local to a region and require a		

rebuild on a regional failure. There is no data stored for this service. | <=12 hours | not applicable |

| Gitaly Cluster | Initially, Gitaly Cluster consists of a single SPOF node and remains so for Cells 1.0. It requires a rebuild in the case of a regional failure. | Unknown | <=1 hr for snapshot restore until WAL is available for restore. [^blueprint-dr] |

| Redis Cluster | Redis is deployed in a single region and requires a rebuild in the case of a regional failure. In flight jobs, session data and cache can not be recovered. | Unknown | not applicable |

| PostgreSQL Cluster | The PostgreSQL cluster is deployed in a single region and requires a rebuild in the case of a regional failure. Recovery is from backups and WAL files. A small amount of data-loss might occur on failover. | Unknown | <=5 minutes |

NOTE:

For data stored in Object storage in Cells multi-region buckets are used. For restoring data due to accidental deletion we rely on object versioning or backups for recovery.

Disaster Recovery Validation

Disaster Recovery for Cells needs to be validated through periodic restore testing.

This recovery should be done on a Cell in the Production environment.

This testing is done once a quarter and is completed by running game-days using the disaster recovery runbook.

Disaster Recovery Requirements

We intend to meet the following requirements for DR.

Backup/Restore will need to meet all requirements while Geo will not protect against application bugs, accidental deletion, or ransomware.

| Requirement | Geo | Backup and Restore |

| --- | --- | --- |

| Data must be stored in different regions than the operating regions of the Cells (us-east1, us-central1). | · | · |

| Employees who support the day-to-day operation of a cell cannot have permission to delete backups or replicated data. | · | · |

| The principal or identity used to run the GitLab application cannot delete backups or replicated data. | · | · |

| The principal or identity used to support Disaster Recovery cannot delete backups or replicated data. | · | · |

| If backups or replicated data can be modified or deleted, the permissions to do so must be restricted to a limited number of operators who can only access backups using an offline credential such as a securely stored MFA token. | · | · |

| Replication/backups must be able to run unattended. | · | · |

| Replication/backups must be monitored and adhere to a specific SLO. | · | · |

| We must be able to configure a retention policy. | N/A | · |

Disaster Recovery Risks

1. The Legacy Cell is not using the same tooling for deployment and operation where the other Cells are. This might split our processes and runbooks and add to our RTO if restoring the

Legacy Cell is needed along with other Cells.

Geo

Supporting infrastructure for Geo will be provisioned using the [GitLab Environment Toolkit] and [GitLab Dedicated Instrumentor].

This will allow operators to configure a Cell to fail over the GitLab application to an alternate region.

Geo Risks

1. The infrastructure provisioning for Geo on GCP is not yet available and is still under development.

Backup and Restore

[Unified Backup] is a single command-line tool that will handle the application backup and recovery needs of GitLab installations across supported reference architectures.

The tool will allow for the backup of application data and its recovery, and is what Cells will use to ensure we meet the full list of Disaster Recovery requirements.

The Unified Backup tooling will allow for backup and restore of the following data:

1. Git repositories
1. Application data
1. Object storage

Git Repositories

Git repositories are stored on disk using GCP's persistent disks.

We do not intend to support Praefect for Cells.

A solution to backup data must be present in addition to features like Gitaly RAFT that provide high availability.

At this time, Cells does not yet support configuring resource policies for disk snapshots.

The GitLab Unified Backup tool will initiate a snapshot of all Gitaly volumes, that is being explored in the disk snapshot integration epic.

- · Region isolation: Snapshots are stored in GCP object storage that is multi-region.
- · Deletion protection: There is no mechanism outside of iam.Deny policies to prevent the deletion of snapshots.

Application Data

Application data is stored in PostgreSQL using CloudSQL in GCP.

Cells enables automated backups when provisioning CloudSQL with a configurable retention window.

The GitLab Unified Backup tool is exploring using on-demand backups, but it is unclear now how this will integrate with Cells.

- · Region isolation: CloudSQL backups are stored in GCP object storage that is multi-region.

- · Deletion protection: There is no mechanism outside of iam.Deny policies to prevent the deletion of a CloudSQL instance and its backups.

Object Storage

Object storage in Google cloud can enable [soft-delete] to allow recovery of accidental or intentional deletions.

Additionally, for a separate backup a [bucket lock] can be added to ensure that objects are not deleted for a retention period.

The GitLab Unified Backup tool is developing an on-demand object storage backup that initiates a copy of all object storage to a separate bucket using the Google Transfer Service.

- · Region isolation: Snapshots are stored in GCP object storage that will be multi-region.
- · Deletion protection: There are multiple mechanisms to prevent deletion, one is soft-delete and the other is a retention lock on the backup bucket.

Metadata/Secrets

There is nothing currently planned for backing up secrets in GitLab Unified Backup.

- · Region isolation: Google secrets manager will store secrets in multiple regions.
- · Deletion protection: Google secrets manager has a version-destroy-ttl option for secrets that is being investigated in this issue

Backup and Restore Risks and Unknowns

1. It is intended that the unified backup tool will be launched inside the toolbox pod, to meet the requirement to run backups unattended there will need to be a way to run the CLI periodically. We are still unsure how the automatic runs will be configured and monitored.

1. It isn't clear yet how Unified Backup will enable unattended restores, for automated testing.

1. We don't yet know how we can monitor a successful backup.

1. The GitLab Unified Backup tool does not provision infrastructure, and we have not yet decided on what additional resources will need to be provisioned for backups.

1. It's unclear at this time what the boundaries are between Unified Backup and Cells backup configuration, like enabling backups for CloudSQL and setting up resource policies for scheduled snapshots.

1. We aren't yet sure how restore will work for Cells which is part of the [discovery for integrating Unified Backup with Cells].

[Cells 1.0 Architecture Overview]: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/doc/architecture/blueprints/cells/iterations/cells-1.0.mdarchitecture-overview>

[Recovery Time Objective]:

<https://en.wikipedia.org/wiki/DisasterrecoveryRecoveryTimeObjective>

[Recovery Point Objective]:

<https://en.wikipedia.org/wiki/DisasterrecoveryRecoveryPointObjective>

[Write-ahead logging]: <https://en.wikipedia.org/wiki/Write-aheadlogging>

[Next Gen Scalable Backup and Restore]: <https://gitlab.com/groups/gitlab-org/-/epics/11577>
[GitLab Environment Toolkit]: <https://gitlab.com/gitlab-org/gitlab-environment-toolkit>
[GitLab Dedicated Instrumentor]: <https://gitlab-com.gitlab.io/gl-infra/gitlab-dedicated/team/architecture/blueprints/dedicated-gcp-introduction.html>
instrumentor-the-dedicated-provisioner
[discovery for integrating Unified Backup with Cells]: <https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues/25736>
[Unified Backup]: <https://gitlab.com/groups/gitlab-org/-/epics/11577>

^blueprint-dr]: See the [DR Blueprint
[^object-storage]: Data are stored on Google Object Storage makes no RPO guarantees for regional failure. At this time, there are no plans to use dual-region buckets which have a 15 minute RPO guarantee.

SECTION: engineering/architecture/design-documents/cells/infrastructure/feature_flags.md

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

Feature flags are a powerful tool that provide SREs and developers a way to control behavior of the application in a rapid manner.
They are also commonly being used in the industry to increase personalisation and as go to market tools e.g. LaunchDarkly.

Currently, with a single production instance, setting of feature flags is an operation that is performed with a single ChatOps command, and immediately rolled out to production with an API call.

With the introduction of multiple cells, engineers setting feature flags in each cell will be unmanageable.

Reference the previous iteration of this ADR:
Cells: Feature Flags

Motivation

With the change of Cells milestones and their iterations, the premise that we will manage feature flags only on the legacy cell and not on any of the cells is no longer true.

Feature flags are commonly used to introduce features that are considered risky, or features that are not considered production ready. In addition, feature flags are regularly used to mitigate incidents.

Feature flags will need to be available for use on Cells before the first cell has a customer.

Goals

This document describes at a high-level how feature flags will be set and rolled out to Cells.

Implementation details are left out of this document. In some cases, options for implementing the design are mentioned.

- Describe the interface for engineers and SREs to set feature flags in Cells.
- Define rollout strategies for feature flags in Cells.

Non-Goals

Other aspects of feature flags, though they might need to be revisited in preparation for Cells, will not be discussed here.

- Visualizing current feature flag state.
- Feature flag lifecycle changes; Limiting number of feature flags, reducing long lived feature flags.

Proposal

Key terms

- ChatOps - ChatOps commands are used to control feature flags on environments.
- Tissue - The Tissue project is used for coordinating changes in Cells.

Overview

Current usage

ChatOps is currently used to set feature flags. A ChatOps command triggers a pipeline in the ChatOps project which sets the feature flag on production using the feature flag API. Feature flags on the legacy cell can continue to be set by this mechanism.

The set of feature flags enabled on the legacy cell and on other cells is expected to be different.

Rollout of feature flags to Cells

One of the main uses of feature flags is to de-risk the release of a new feature. A feature flag is usually enabled for a small group of customers (including internal customers) who are willing to test out the feature. Any bugs reported are fixed, and then the feature flag is enabled for a larger group of customers. This process

repeats until the feature flag is enabled for all customers, the feature is considered stable and the feature flag can be removed.

While feature flags are useful for rapidly changing behavior of the application, having too many different combinations of feature flags can cause interactions that produce unforeseen behavior. It can also make it difficult to predict how a new change will behave and can make incident management more challenging.

We need to allow feature flags to be enabled for small groups of customers, and at the same time maintain the number of different feature flag configurations in production at a manageable level. We can leverage the ring architecture (introduced in Application Deployment with a Cellular Architecture) to do this.

Each ring represents a different groups of customers. As we move from inner to outer rings, each consecutive ring is supposed to be less risky for the customer. Customers who are accepting of higher risk, and desiring the latest product updates will be on inner rings, while customers desiring stability will be on outer rings. Each ring can be viewed as a "layer of risk". To reduce the number of different feature flag configurations that need to be supported in production, while still allowing feature flags to be rolled out to small groups of customers at a time, feature flags can be rolled out to a ring at a time, starting with the innermost ring.

Rollout of feature flag changes to a ring should not be allowed if there is an active incident affecting that ring or if metrics indicate that the ring is unhealthy.

Interface for controlling feature flags in Cells

ChatOps can be used to rollout feature flags to Cells in the short term.

ChatOps can communicate with Tissue to set feature flags on Cells. ringctl can be used to create a patch for modifying feature flag gate values. Tissue's change management system will then rollout the feature flag change to the rings.

Support for feature flag actors

Feature flags on Cells will support the same actors (project, group, user, currentrequest, :instance) that are supported on production today.

If a feature flag needs to be set for a particular actor, it should be set for that actor on every ring. This ensures that no matter which cell the actor is present on, the feature flag will behave as expected.

In addition, the organization model will also be enabled for use as a feature flag actor (<https://gitlab.com/gitlab-org/gitlab/-/issues/498238>).

Short term

Iteration 1

ChatOps will provide an interface for engineers to change a feature flag on Cells.

ChatOps communicates with Tissue, and Tissue sets the feature flag on every cell.

Success or failure of the operation will be communicated via comments on the rollout issue as well as issues in the feature flag log project.

If a feature flag needs to be turned off due to an incident, it will have to be turned off for all cells.

As a feature flag change is rolled out, create issues on the feature flag log project.

Also add a comment to the feature flag rollout issue.

When the current state of a feature flag is queried, Tissue will query each cell.

Epic: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1423>

Iteration 2

ChatOps commands will provide an interface for targeting individual cells. Targeting a cell will only be allowed if the cell is in the quarantine ring. In all other cases, the feature flag change should be propagated to all cells.

The current state of feature flags in each cell should be cached. When queried for the current state of a feature flag, the cache can be used instead of reaching out to every cell.

The cache can be refreshed when feature flag changes are rolled out. It should also be refreshed periodically, to keep the cache updated in case manual changes are made to feature flags in a cell.

Epic: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1448>

Long term

Rollout of feature flag changes should be automated as much as possible. Metrics and logs should be defined for each feature flag, to allow Tissue to automate the rollout. The initiator indicates upto which ring the change should be rolled out. Rollout will start with ring 0. It will skip ring 1 (legacy cell), and continue with ring 2.

A feature flag service can be considered as we start to add more functionality for feature flags. The cost of building and running a service will have to be weighed against the benefits and risk of overloading Tissue with feature flag functionality.

The rollout methods described here are for the "derisking" type of feature flags. We can provide different rollout capabilities for other types of feature flags, such as experiment feature flags, feature flags for A/B testing, etc. For example, feature flags for A/B testing could be allowed only on certain rings.

But this restriction would not apply to the derisking type of FFs.

Gradual Rollout

Gradual rollout will require engineers to define metric and log queries, and thresholds that can be used to determine if the feature being gated is behaving in a healthy manner.

- The initiator indicates upto which ring the rollout should occur.
- After rollout to a ring is complete, rollout will pause. After a period of time, if the metrics (FF metrics as well as general metrics) or logs indicate that the ring that the FF was ro